

Lernziele:

- Vertiefung: Assembler-Code für Funktionsdefinitionen und Funktionsaufrufe

Vorbereitung: Compiler aktualisieren

1. Wechselt in das Verzeichnis **abc-llvm** mit dem Source-Code für den Compiler.
2. Führt das Kommando **git pull** aus.
3. Führt das Kommando **make** aus.
4. Führt das Kommando **sudo make install** aus.

Zielsetzung

Um Funktionen zu unterstützen, benötigen wir im Code-Generator ein paar Wrapper-Funktionen, die uns Assembler-Code für folgende Pattern-Bausteine erzeugen. Zum Beispiel:

- **genInit**: Initialisiert den Stackpointer und ruft die Funktion **main** auf.
- Für die Callee-Pattern:
 - **genBeginFunctionDefinition**: Erzeugt ein Funktionslabel und den Prolog.
 - **genEndFunctionDefinition**: Erzeugt den Epilog.
 - **genFetchParameter**: Lädt einen Funktionsparameter in ein Register.
- Für die Caller-Pattern:
 - **genPrepareFunctionCall**: Reserviert Platz auf dem Stack für die Parameterübergabe an die Funktion.
 - **genStoreArgument**: Übergibt ein Argument als Parameter an die Funktion.
 - **genExecuteFunctionCall**: Springt zur Funktion und räumt anschließend den Platz für die Parameterübergabe vom Stack.

Das Ganze ist nicht besonders kompliziert, aber es erfordert viel Schreibarbeit. Im Prinzip muss jede Funktion nur die Assembler-Blöcke ausgeben, die wir in Session 18 als Pattern festgelegt haben. Zum Beispiel sieht bei mir die Implementierung von **genBeginFunctionDefinition** so aus:

```
fn genBeginFunctionDefinition(name: -> UStr)
{
    printf("%s:\n", name);
    printf("\tmovq    %%3,        (%%SP)\n");
    printf("\tmovq    %%FP,        8(%%SP)\n");
    printf("\tmovq    %%SP,        %%FP\n");
    printf("\t\t// begin function body\n");
}
```

Die Funktion erwartet als Parameter den String name. In der erzeugten Ausgabe wird der Name für das Label verwendet. Die restliche Ausgabe ist der Prolog, den ich ohne weiteres Nachdenken aus Session 18 übernommen habe. Hier ist das Kniffligste zu wissen, dass man im Formatstring mit \t ein Tab erzeugen kann und um ein einzelnes '%' auszugeben, zwei aufeinanderfolgende Prozentzeichen verwenden muss.

Schritt 1

Mit einem Testprogramm xtest_genfn.abc sollen die neuen Funktionen im Code-Generator getestet werden. Im ersten Versuch soll damit manuell Code erzeugt werden, der folgendem Programm entspricht:

```
fn foo()
{
}

fn main()
{
    foo();
}
```

Die Parameterübergabe und der Zugriff auf die Parameter wird zunächst ausgelassen. Gebt dazu das folgende Listing ein:

```

1 @ "gen.hdr"
2
3 fn main()
4 {
5     genInit();
6     genBeginFunctionDefinition("foo");
7     genEndFunctionDefinition();
8
9     genBeginFunctionDefinition("main");
10
11     genPrepareFunctionCall(0);
12     genReleaseReg(genExecuteFunctionCall("foo", 0));
13
14     genEndFunctionDefinition();
15 }

```

- In den Zeilen 6 und 7 wird die Funktion **foo** definiert, die im Funktionsrumpf keine Anweisungen enthält. In den Zeilen 9 bis 14 wird die Funktion **main** definiert. Diese enthält den Funktionsaufruf von **foo**.
- Ein Funktionsaufruf beginnt damit, dass man auf dem Stack Platz für die Parameterübergabe schafft. Das passiert mit dem Aufruf von **genPrepareFunctionCall** in Zeile 11. Hier muss bekannt sein, wie viele Parameter insgesamt übergeben werden. In diesem Fall wird kein Parameter übergeben.
- Der Aufruf der Funktion findet mit **genExecuteFunctionCall** in Zeile 12 statt. Hier wird der Name der Funktion übergeben und ebenfalls die Anzahl der Parameter, sodass nach dem Aufruf alle übergebenen Argumente wieder vom Stack entfernt werden können.

Bei uns hat jede Funktion einen Rückgabewert. Dies ist stets der Wert der letzten Anweisung, die in der Funktion ausgeführt wurde. Deshalb hat **genExecuteFunctionCall** als Rückgabewert ein Register, in dem der Rückgabewert anschließend geladen wurde.

In **gen.hdr** könnt ihr am Ende folgende Deklarationen hinzufügen:

```

/-- handle functions
extern fn genInit();

extern fn genBeginFunctionDefinition(name: -> UStr);
extern fn genEndFunctionDefinition();

extern fn genPrepareFunctionCall(numArgs: int);
extern fn genExecuteFunctionCall(name: -> UStr, numArgs: int): GenReg;

```

In **gen.addr** könnt ihr am Ende zunächst Stubs dazu einfügen:

```

/-- handle functions
fn genInit() {}

fn genBeginFunctionDefinition(name: -> UStr) {}

fn genEndFunctionDefinition() {}

fn genPrepareFunctionCall(numArgs: int) {}

fn genExecuteFunctionCall(name: -> UStr, numArgs: int): GenReg
{
    local reg: GenReg = getReg();
    return reg;
}

```

Aufgabe

Schreibt das Testprogramm **xtest_genfn.abc** und ergänzt **gen.hdr** und **gen.abc** wie angegeben. Danach sollte sich alles kompilieren und linken lassen.

Schritt 2

Ab jetzt wird das Register %3 eine Sonderbedeutung haben. Bei einem Funktionsaufruf wird in diesem Register die Rücksprungadresse hinterlegt. Die Funktion **getReg** sollte also so angepasst werden, dass sie nie den Wert 3 zurückgibt.

Die Implementierungen der meisten Funktionen sind reine Fleißarbeit und deshalb zeige ich gerne meinen Code, um euch die Arbeit etwas zu erleichtern. Es schadet natürlich nicht, den Code genauer anzuschauen und nochmals im Kopf die Bedeutung des ausgegebenen Assemblercodes zu wiederholen. Die Funktion genInit bearbeiten wir später, hier mein Code für die restlichen neuen Funktionen:

```
fn genBeginFunctionDefinition(name: -> UStr)
{
    printf("%s:\n", name);
    printf("\tmovq    %%3,        (%%SP)\n");
    printf("\tmovq    %%FP,        8(%%SP)\n");
    printf("\tmovq    %%SP,        %%FP\n");
    printf("\t// begin function body\n");
}

fn genEndFunctionDefinition()
{
    printf("\t// store return value\n");
    printf("\tmovq    %%3,        retval(%%FP)\n");
    printf("\t// end function body\n");
    printf("\tmovq    %%FP,        %%SP\n");
    printf("\tmovq    8(%%SP),        %%FP\n");
    printf("\tmovq    (%%SP),        %%3\n");
    printf("\tret     %%3\n");
}

fn genPrepareFunctionCall(numArgs: int)
{
    printf("\tsub     8*(3 + %d),    %%SP,        %%SP\n", numArgs);
}

fn genExecuteFunctionCall(name: -> UStr, numArgs: int): GenReg
{
    local reg: GenReg = getReg();
    printf("\tcall    %s,        %%3\n", name);
    printf("\tmovq    retval(%%SP), %%d\n", reg);
    printf("\tadd     8*(3 + %d),    %%SP,        %%SP\n", numArgs);
    return reg;
}
```

Aufgabe

Implementiert die angegebenen Funktionen. Stellt sicher, dass sich das Programm übersetzen und ausführen lässt.

Schritt 3

Unsere ISA unterstützt aktuell noch nicht alle Instruktionen, die wir für Funktionen brauchen. Ergänzt in **simple.isa** zunächst Befehlsformate. Insgesamt benötigen wir folgende Formate:

S20_R	(OP u 8) (dest u 4) (imm s 20)
S16_R_R	(OP u 8) (z u 4) (y u 4) (imm s 16)
R_R_R	(OP u 8) (z u 4) (y u 4) (x u 4)
MEM	(OP u 8) (data u 4) (addr u 4) (offset s 16)

Neu ist das Format S16_R_R. Dieses wird zum Beispiel bei der neuen Instruktion add (und analog bei sub) verwendet, um ein Immediate-Value auf ein Register zu addieren und das Ergebnis in ein Zielregister zu schreiben.

Fügt am Ende von simple.isa noch folgende Instruktionsbeschreibungen ein:

```
0x30 S16_R_R
:   add imm, %y, %z
    ulm_add64(imm, ulm_regVal(y), z);

0x31 S16_R_R
:   sub imm, %y, %z
    ulm_sub64(imm, ulm_regVal(y), z);

0x32 S20_R
:   call imm, %dest
    ulm_absJump(imm, dest);

0x33 R_R_R
:   ret      %x
    ulm_absJump(ulm_regVal(x), 0);

0x34 R_R_R
:   movq %x, %y
    ulm_add64(ulm_regVal(x), ulm_regVal(0), y);

0x35 R_R_R
:   putc     %x
    ulm_printChar(ulm_regVal(x));

0x36 R_R_R
:   getc     %x
    ulm_setReg(ulm_readChar(), x);
```

Aufgabe

Erzeugt eine neue virtuelle Maschine und einen neuen Assembler für die erweiterte ISA:

- Löscht das Unterverzeichnis simple mit dem Kommando **rm -rf simple**
- Erzeugt die Tools mit **ulm-generator --install simple.isa**

Testet, ob sich die Ausgabe von **xtest_genfn** damit übersetzen lässt.

```
MCL:s25 lehn$ ./xtest_genfn > foo.s
MCL:s25 lehn$ ulmas foo.s
```

Schritt 4

Implementiert im letzten Schritt die Funktion genlnit, sodass sie folgenden Text ausgibt:

```
.equ SP,      1
.equ FP,      2
.equ retval,  16
.text
load 0,       %SP
sub    8*(3 + 0), %SP, %SP
call   main,   %3
movq   retval(%SP), %4
add    8*(3 + 0), %SP, %SP
halt   %4
```

Wenn ihr euch den Assemblercode genauer anschaut, seht ihr, dass man hier natürlich nicht alles mit printf-Anweisungen erzeugen muss (kann man aber). Anschließend sollte sich der erzeugte Assemblercode in ein ausführbares Programm übersetzen lassen. Führt das dann im Debugger aus, um zu prüfen, dass die Funktionsaufrufe wie gewünscht ausgeführt werden.